

Development Schedule

Cloud AI Monitor Platform – Hybrid Architecture

CIRES Technologies

Supervised by: IBRAHIM EL HANNAOUI

Prepared by: Farouk Toujgani & Redouane Meriche

July 2026

Project Overview

Target Technologies:

- **Backend:** Python 3.11, FastAPI (Clean Architecture), SQLAlchemy, PostgreSQL, APScheduler, WebSockets.
- **Frontend:** Node.js 18+, Vite, React, TypeScript, Axios, Recharts.
- **Infrastructure:** OpenStack (VMs), OpenShift / Kubernetes (Pods).

Week 1: Backend Foundations, Data Collection, and AI

Day 1: Scoping and Initial Architecture

- Set up the development environment (Python 3.11, Node.js 18+).
- Initialize FastAPI using *Clean Architecture* principles (Repository Pattern, Service Layer).
- Configure environment variables (`.env`) to seamlessly toggle between Mock and Production modes.

Day 2: Data Models and Ingestion Layer

- Create SQLAlchemy ORM models: Resources (VMs, Pods), Metrics, Alerts, and Logs.
- Configure the PostgreSQL database and Redis cache (preparing for real-time state management).
- Implement the Ingestion Buffer to normalize heterogeneous incoming telemetry data.

Day 3: OpenStack Collector and Abstraction

- Develop the OpenStack client (`openstack_client.py`).
- Implement the Mock Service to generate simulated VMs and metrics.
- Create API routes (e.g., `GET /api/vms`) utilizing dependency injection.

Day 4: OpenShift / Kubernetes Collector

- Develop the Kubernetes client (`openshift_client.py`).
- Retrieve pod statuses, namespaces, and restart counts (e.g., `CrashLoopBackOff`).
- Implement the Mock Service for pod simulation.

Day 5: Scheduling and Real-Time Capabilities

- Configure `APScheduler` for continuous data collection (30-second intervals).
- Implement *upsert* logic for persisting metrics in the database.
- Establish the WebSocket Gateway to handle real-time data broadcasting.

Day 6: Intelligent Alert Engine

- Implement the evaluation logic (`evaluate_vm`, `evaluate_pod`).
- Configure anomaly detection thresholds (CPU/RAM limits, Failed/Error statuses).
- Manage the alert lifecycle (Creation, deduplication, automatic resolution).

Day 7: NOC AI Decision Agent

- Develop `analyze_alert` to calculate the *Health Score*.
- Integrate the core evaluation formula:

$$H = 100 - (\alpha \cdot m + \beta \cdot l + \gamma \cdot t)$$

- Classify alerts, determine confidence levels, and generate operational recommendations (escalate, investigate, watch, suppress).

Week 2: Auto-remediation, Frontend, and Integration

Day 8: AIOps Auto-remediation Agent

- Develop `execute_remediation` targeting specific actions (`extend_storage`, `scale_memory`, `restart_workload`).
- Implement security guardrails and the `dry_run` simulation mode.
- Persist action statuses (`recommended`, `applied_mock`, `applied`).

Day 9: Frontend Structure and Connectivity

- Initialize the Vite + React + TypeScript application.
- Build the API client using Axios and configure the CORS proxy.
- Develop the central Dashboard: Map KPIs and connect to the WebSocket stream for dynamic refreshing.

Day 10: Infrastructure Views (VMs & Pods)

- Build the `VMsPage`: Display statuses, flavors, hosts, and consumption charts using Recharts.
- Build the `PodsPage`: Implement namespace filtering and anomalous restart detection.
- Manage loading states and error boundaries.

Day 11: Alerts View and AI Interactions

- Build the `AlertsPage`: Design the data table for active and historical alerts.
- Custom UI: Visually integrate the NOC Agent's recommendation cards and confidence badges.
- Add interactive buttons for manual resolution and auto-remediation approval.

Day 12: End-to-End Testing and Interchangeability

- Test the *Mock / Production* toggle utilizing environment variables.
- Execute backend unit tests (rules engine) and frontend tests (component rendering).
- Validate the full pipeline: Collector → Engine → AI Agents → WebSocket → React UI.

Day 13: Configuration and Audit Logging

- Build the settings page to configure AI thresholds and AIOps security limits.
- Display the audit trail (logging) for all remediation actions.
- Implement advanced handling for API timeouts and WebSocket disconnections.

Day 14: Review, Documentation, and Final Deliverables

- Conduct a final code review and clean up the repository.
- Write comprehensive documentation in `README.md` (deployment steps, environment variables).
- Prepare the demonstration scenario for the final presentation (Simulated failure → AI Analysis → Remediation).